VISUAL NOTES • COMPLETE GUIDE

AI Testing: Context, Independence and Observability

Playwright MCP vs CLI, Mutation Testing and AI Agents Flying Blind

09:57

- 16 substantive visual(s), selected by content
- Repeated builds, presenter-only shots, and filler excluded
- Output length expands or contracts with the source material
- Claims, uncertainty, and caveats are labeled clearly

https://www.youtube.com/watch?v=sTeHIfcv7yE

MCP and CLI impose different context costs



NOTES

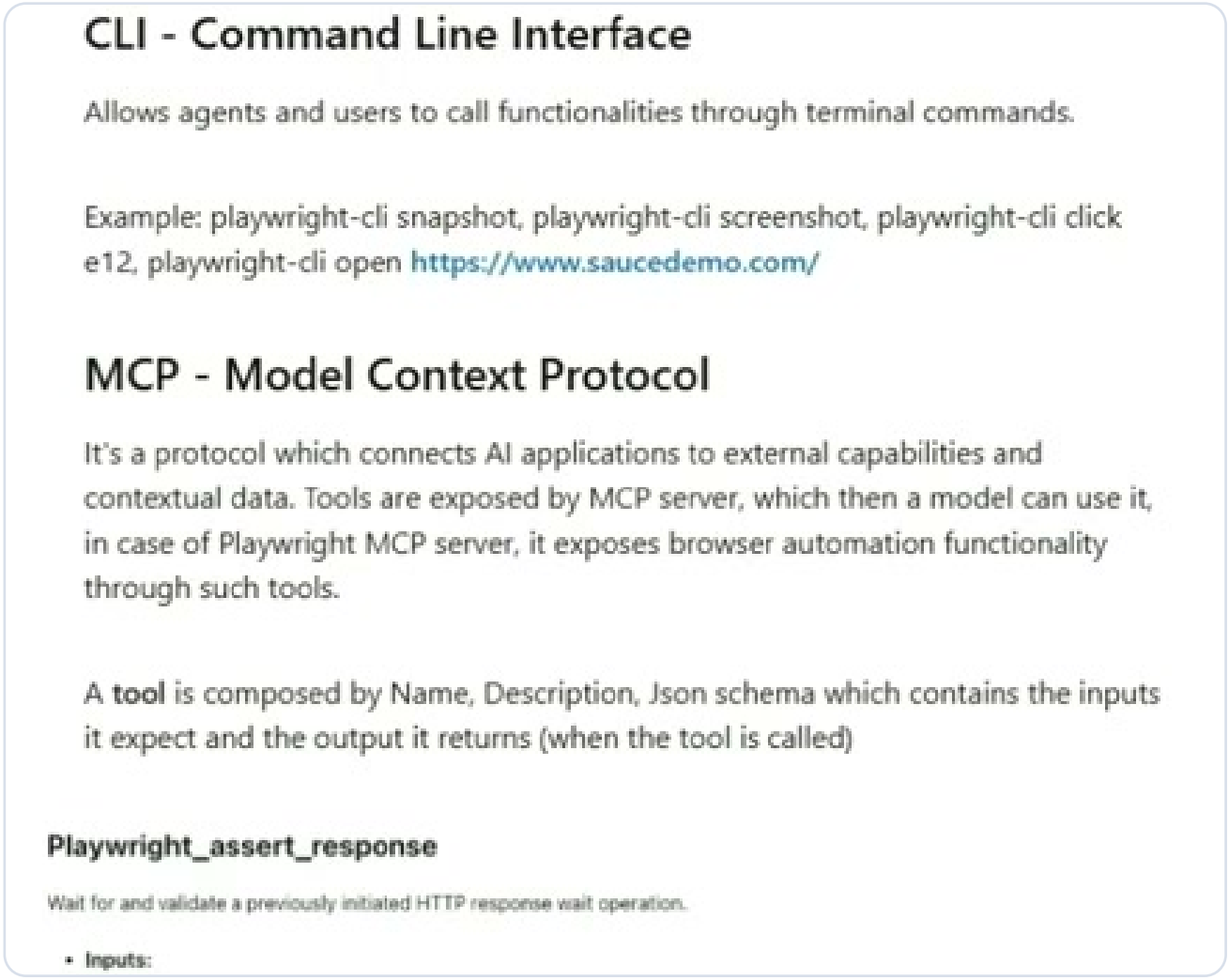
- Playwright MCP places accessibility snapshots directly in tool results, while the CLI can return a path to a snapshot file.
- Repeated inline snapshots may consume context that could otherwise support reasoning and code generation.
- Both interfaces expose similar browser capabilities through different interaction and context-management models.
- This is a practitioner comparison, not an independently reproduced benchmark.

CORE IDEA

Choose the browser interface whose context behavior fits the workload, not the fashionable protocol.

Source: https://www.youtube.com/watch?v=sTeHIfcv7yE

CLI and MCP efficiency depends on the task



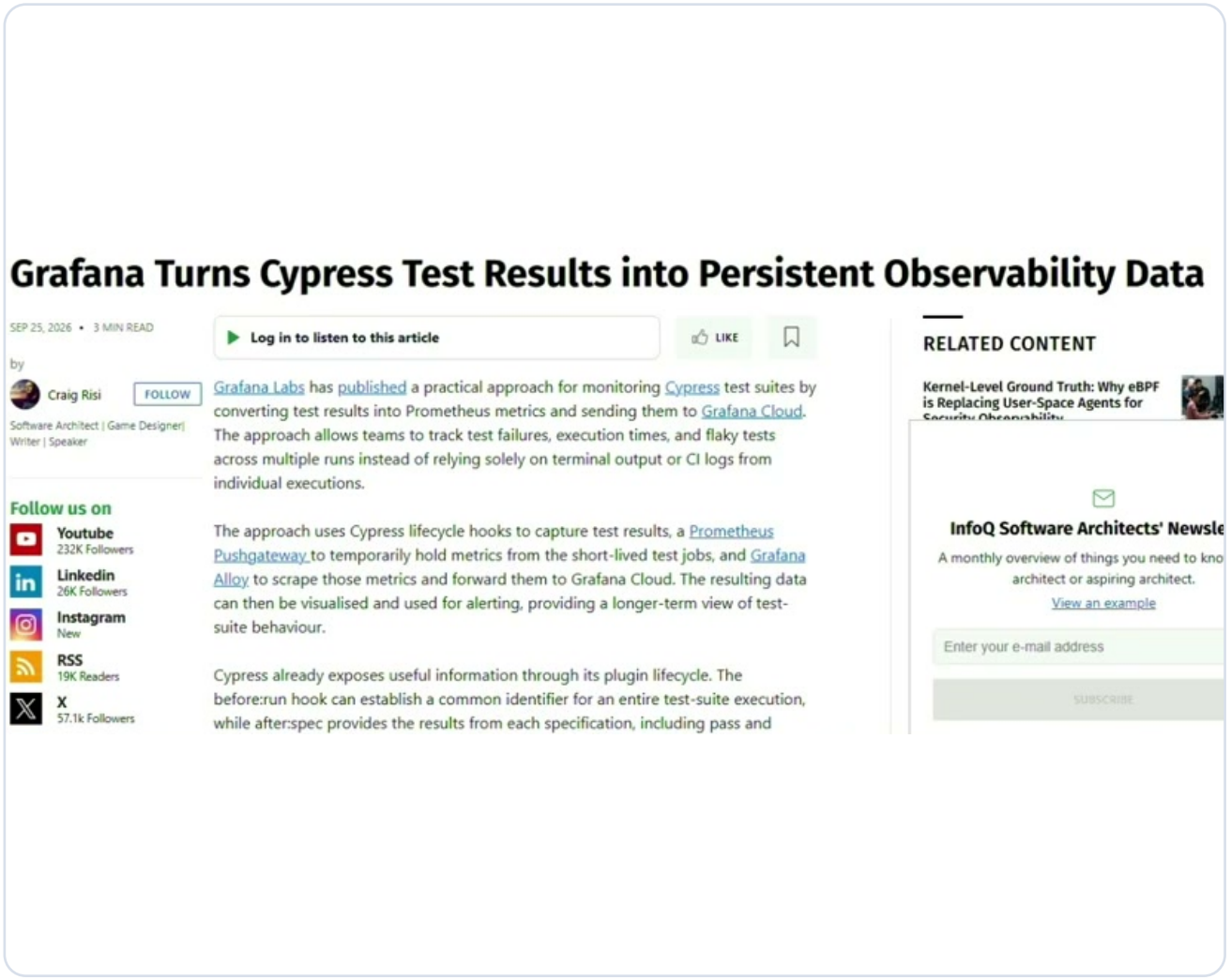
NOTES

- CLI automation uses terminal commands for snapshots, screenshots, clicks, and navigation.
- MCP exposes structured tools with names, descriptions, input schemas, and outputs.
- Reported existing-test usage favored CLI at 1.2 credits versus MCP at 1.5 credits.
- A reported exploratory flaw-finding task reversed the result: MCP used 0.6 credits versus CLI at 5.3.

CORE IDEA

File-oriented coding and exploratory browser work can reward different interfaces.

CI test output can become persistent telemetry



Grafana Turns Cypress Test Results into Persistent Observability Data

SEP 25, 2026 • 3 MIN READ

by

Craig Risi FOLLOW

Software Architect | Game Designer| Writer | Speaker

Follow us on

Youtube 232K Followers

Linkedin 26K Followers

Instagram New

RSS 19K Readers

X 57.1k Followers

Log in to listen to this article LIKE

Grafana Labs has published a practical approach for monitoring Cypress test suites by converting test results into Prometheus metrics and sending them to Grafana Cloud. The approach allows teams to track test failures, execution times, and flaky tests across multiple runs instead of relying solely on terminal output or CI logs from individual executions.

The approach uses Cypress lifecycle hooks to capture test results, a Prometheus Pushgateway to temporarily hold metrics from the short-lived test jobs, and Grafana Alloy to scrape those metrics and forward them to Grafana Cloud. The resulting data can then be visualised and used for alerting, providing a longer-term view of test-suite behaviour.

Cypress already exposes useful information through its plugin lifecycle. The before:run hook can establish a common identifier for an entire test-suite execution, while after:spec provides the results from each specification, including pass and

RELATED CONTENT

Kernel-Level Ground Truth: Why eBPF is Replacing User-Space Agents for Security Observability

InfoQ Software Architects' Newsle

A monthly overview of things you need to kno architect or aspiring architect.

View an example

Enter your e-mail address

SUBSCRIBE

NOTES

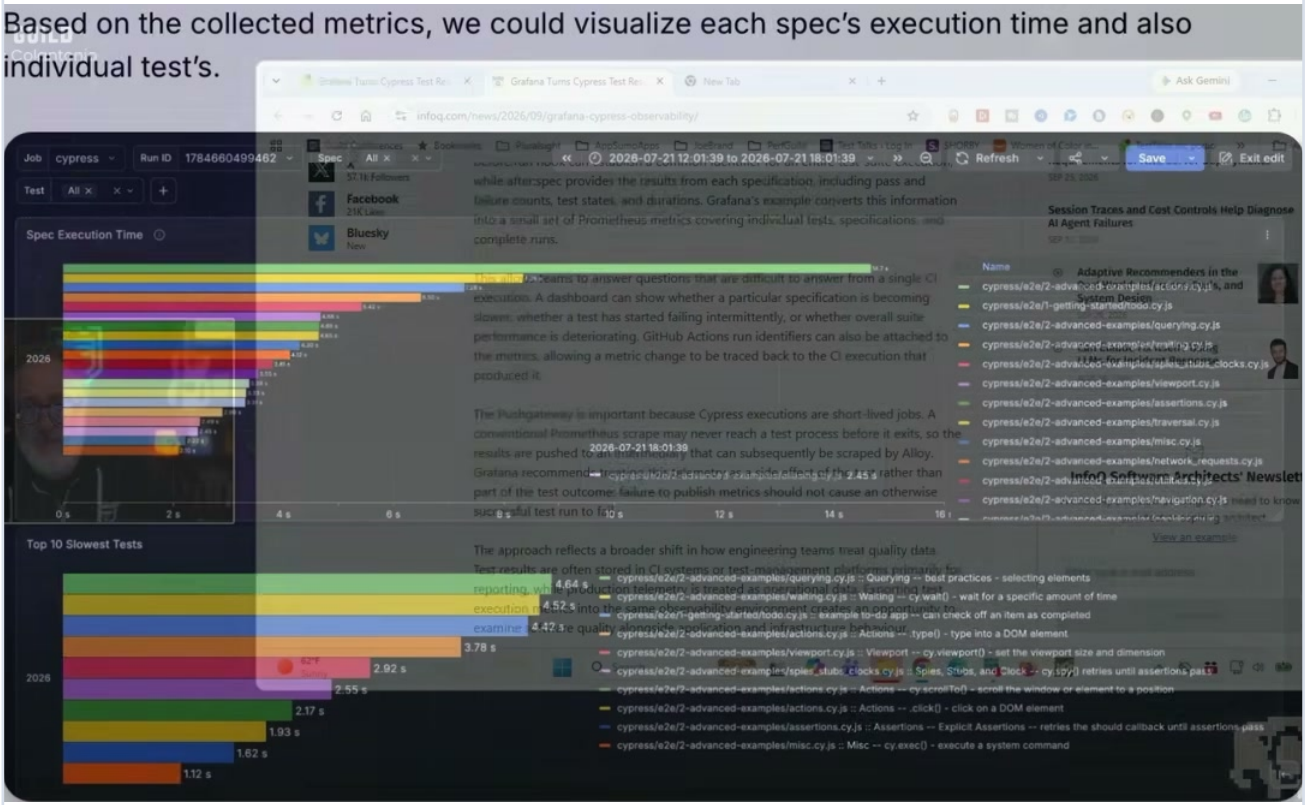
- Cypress lifecycle hooks capture a run identifier plus pass, failure, and duration data per specification.
- A Prometheus Pushgateway retains metrics from short-lived CI jobs.
- Grafana Alloy forwards the metrics to Grafana Cloud for historical analysis.
- A GitHub Actions run ID links trend changes back to the originating CI execution.

CORE IDEA

Persist ephemeral test results so teams can inspect trends instead of isolated CI runs.

Source: https://www.youtube.com/watch?v=sTeHIfcv7yE

Historical timing exposes slow and unstable tests



NOTES

- The dashboard compares execution time across Cypress specification files and ranks slow individual tests.
- Filters cover job, run ID, specification, test, and time range.
- Historical timing can reveal gradual slowdowns and intermittent regressions across runs.
- The screenshot demonstrates a dashboard; it does not prove the pipeline guarantees flaky-test detection.

CORE IDEA

Test telemetry becomes useful when timing and failures can be compared across many runs.

Semantic assertions tolerate valid variation



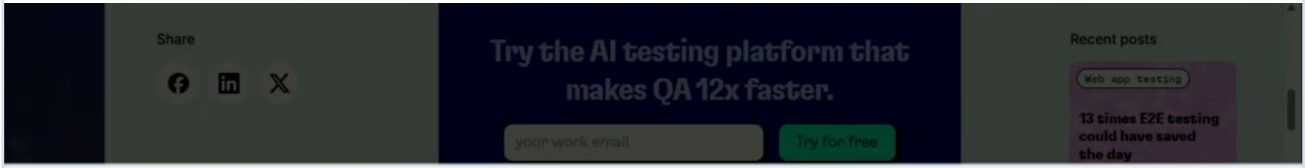
NOTES

- Nondeterministic systems can produce valid language or navigation paths that differ between runs.
- Jev is described as answering typed questions with probabilistic decisions from supplied state.
- The model handles narrow semantic judgments while workflow and business requirements remain in code.
- Probabilistic evaluation still needs uncertainty thresholds, failure handling, and deterministic side-effect checks.

CORE IDEA

Use semantic evaluation for variable outputs without making the whole test probabilistic.

Language variation and route variation need different tools



- `ai.expect(...).toSatisfy(...)` handles variation in generated language.
- `ai.act(...)` handles variation in the route to a known state.

NOTES

- ai.expect(...).toSatisfy(...) checks whether variable wording meets a semantic requirement.
- ai.act(...) searches for a route to a known state when UI navigation changes.
- Step and time limits constrain how far model-driven navigation may search.
- The two primitives solve different uncertainty types and should not be treated as interchangeable.

CORE IDEA

Separate uncertainty in language from uncertainty in navigation, then constrain each explicitly.

Let wording vary while requirements stay fixed

Let the words vary, not the requirement

Exact text assertions work when the words themselves are the requirement. They become brittle when generated language can express the same valid outcome in many ways. Adding every accepted string eventually turns the test into a catalog of phrasing rather than a description of what the product should do.

With `toSatisfy`, the test states the requirement:

```
import { expect } from '@qawolf/flows/web';
```

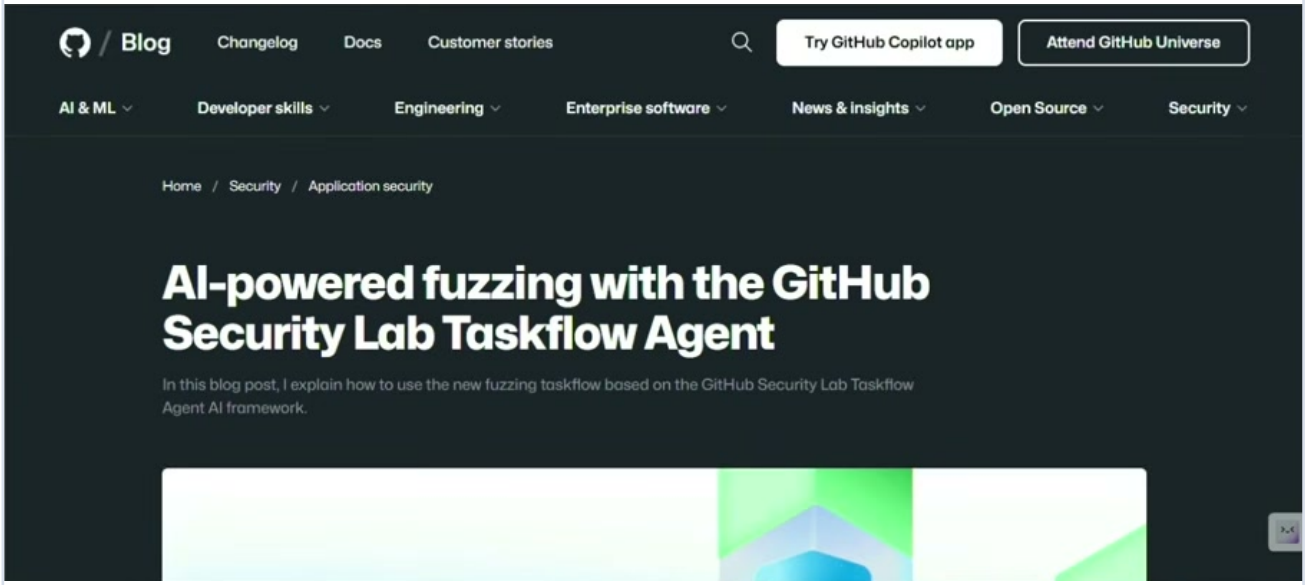
NOTES

- Exact-string assertions are correct when exact wording is itself the requirement.
- They become brittle when many phrasings express the same valid result.
- Enumerating accepted strings turns the requirement into a maintenance-heavy phrase catalog.
- Structured effects such as HTTP responses and ticket routing still require deterministic assertions.

CORE IDEA

Allow linguistic flexibility, but verify business outcomes objectively.

An agent can automate the fuzzing feedback loop



NOTES

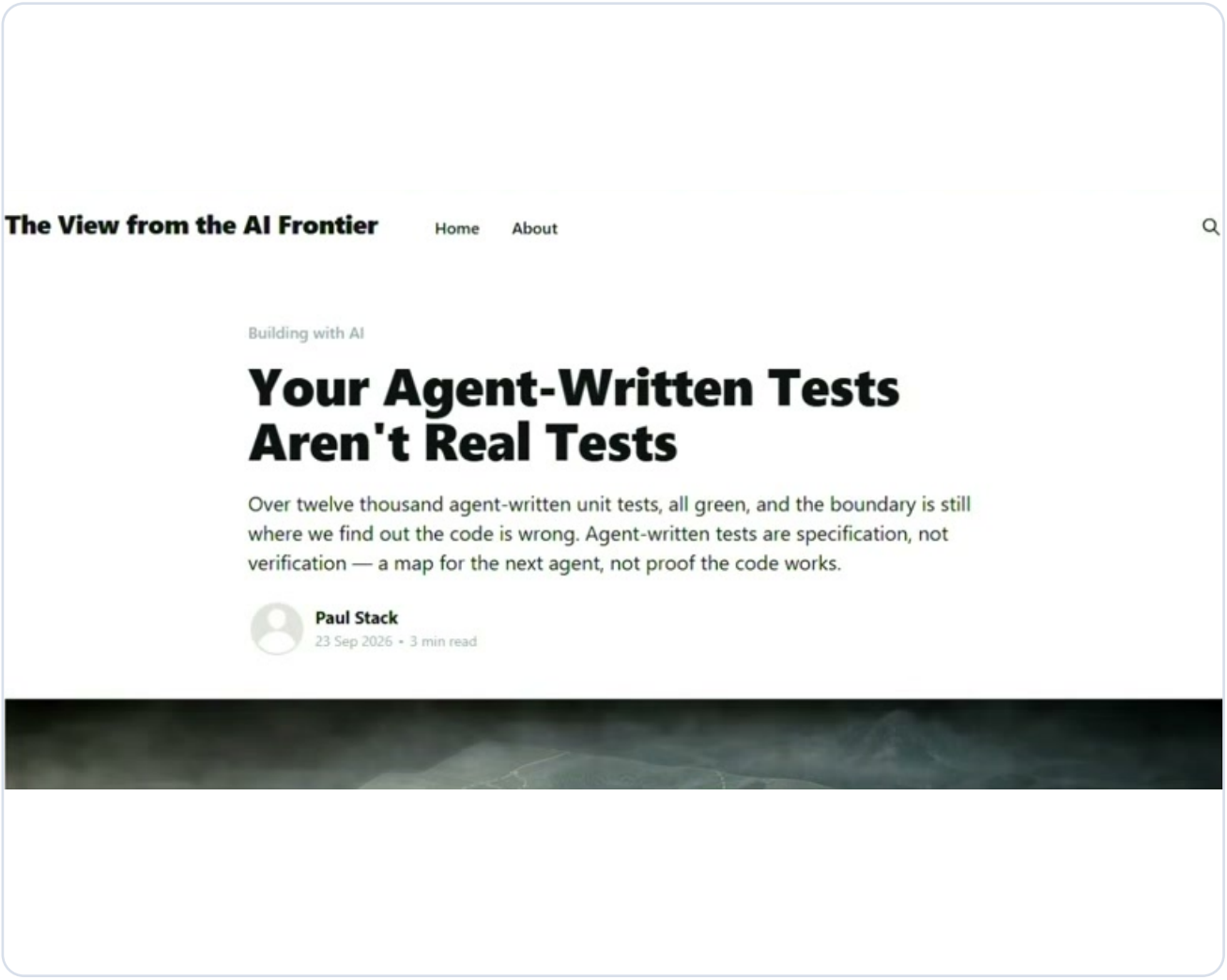
- The open-source workflow targets C and C++ repositories.
- It finds entry points, writes harnesses, runs AFL++, reads coverage, and revises the harness.
- Each iteration doubles the time budget and stops after two rounds add less than 1% line coverage.
- Crash verdicts and suggested fixes remain subject to human review because agent conclusions can be wrong.

CORE IDEA

Automate repetitive fuzzing work while keeping vulnerability judgments and fixes reviewable.

Source: https://www.youtube.com/watch?v=sTeHIfcv7yE

Green agent-written tests may share the code’s blind spot



NOTES

- The reported project had roughly 12,000 green agent-written unit tests.
- Acceptance tests reportedly ran about 2,000 times and found defects in 80 runs.
- Agents generated both the implementation and its unit tests.
- These project figures are reported evidence and should not be generalized without underlying data.

CORE IDEA

A large green suite is weak evidence when code and tests come from the same reasoning process.

Source: https://www.youtube.com/watch?v=sTeHIfcv7yE

The same mistaken interpretation can appear twice

caught real defects — not flaky tests, not environment issues, but genuine failures in code that passed through the entire unit test suite without a problem. Over twelve thousand unit tests, all green, and the boundary is still where we find out the code is wrong.

The same understanding written twice

At Swamp, every line of code is written by agents. No human touches the implementation. We required agents to write unit tests for everything, and they did. You can watch it happen on every PR: the tests change alongside the code, written as an afterthought once the implementation is done. Sometimes the agent does not even bother making them pass, we have seen test failures in CI because the agent simply did not care enough about the tests to keep them green.

Both artefacts — the function and its unit test — encode the same interpretation of the requirement. If the agent misunderstood what you wanted, the test encodes that misunderstanding and the implementation satisfies it. Chen et al. forced six LLMs to write tests across five hundred SWE-bench Verified tasks and

NOTES

- Implementation and unit tests may encode the same interpretation of a requirement.
- If that interpretation is wrong, the code can satisfy the test while violating the actual need.
- Tests written after implementation are especially likely to mirror current behavior.
- A coding loop may rewrite a failing check because modifying the test is the shortest route back to green.

CORE IDEA

Duplicating one interpretation does not create independent verification.

Verification power comes from independent constraints

When an agent does hit a failing test during its loop, the fastest path to green is rewriting the test. In our workflow, that is exactly what happens. A study of over 86,000 agent-authored test patches found that 80.2% contained weak or no explicit oracle signals. And SWE-Gate showed what that means in practice: among patches that passed their functional tests, 34% still failed to satisfy the full requirements derived from real pull request reviews.

Why independence matters

Developers spent a long time working out how to manage this problem. TDD was the most disciplined answer: write the test first, commit to an interface and a set of expectations before the implementation exists. In disciplined TDD, changing a failing test represented a conscious change to the specification. The developer had to decide: is my implementation wrong, or is my specification wrong? That independence — the test as a fixed constraint — is what gave unit tests their verification power.

Agents cannot replicate this. An agent has its interpretation of the requirement

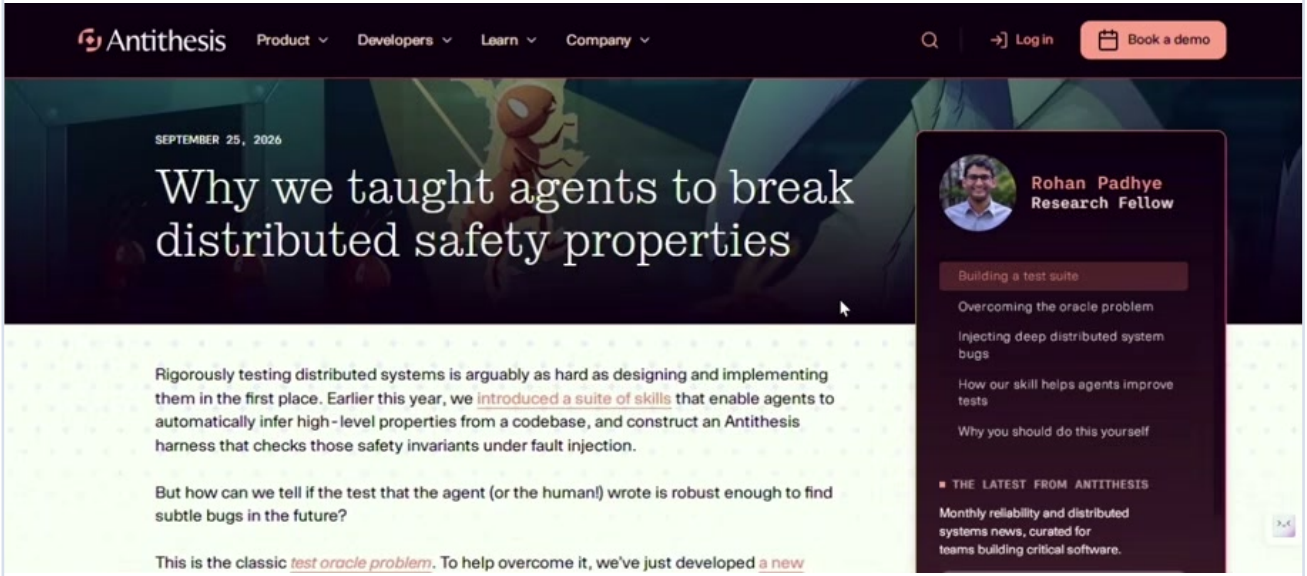
NOTES

- Test-driven development establishes expectations before implementation and treats later changes as specification decisions.
- Release gates can include architectural fitness, property, contract, and black-box tests the coding agent cannot rewrite.
- The visible source reports 80.2% of 86,000 agent-authored test patches had weak or absent oracle signals.
- The transcript says 8.2%; the discrepancy remains unresolved and the figure should not be repeated without checking the cited study.

CORE IDEA

Keep critical checks outside the implementation agent's authority.

Mutation testing asks whether a property can fail



NOTES

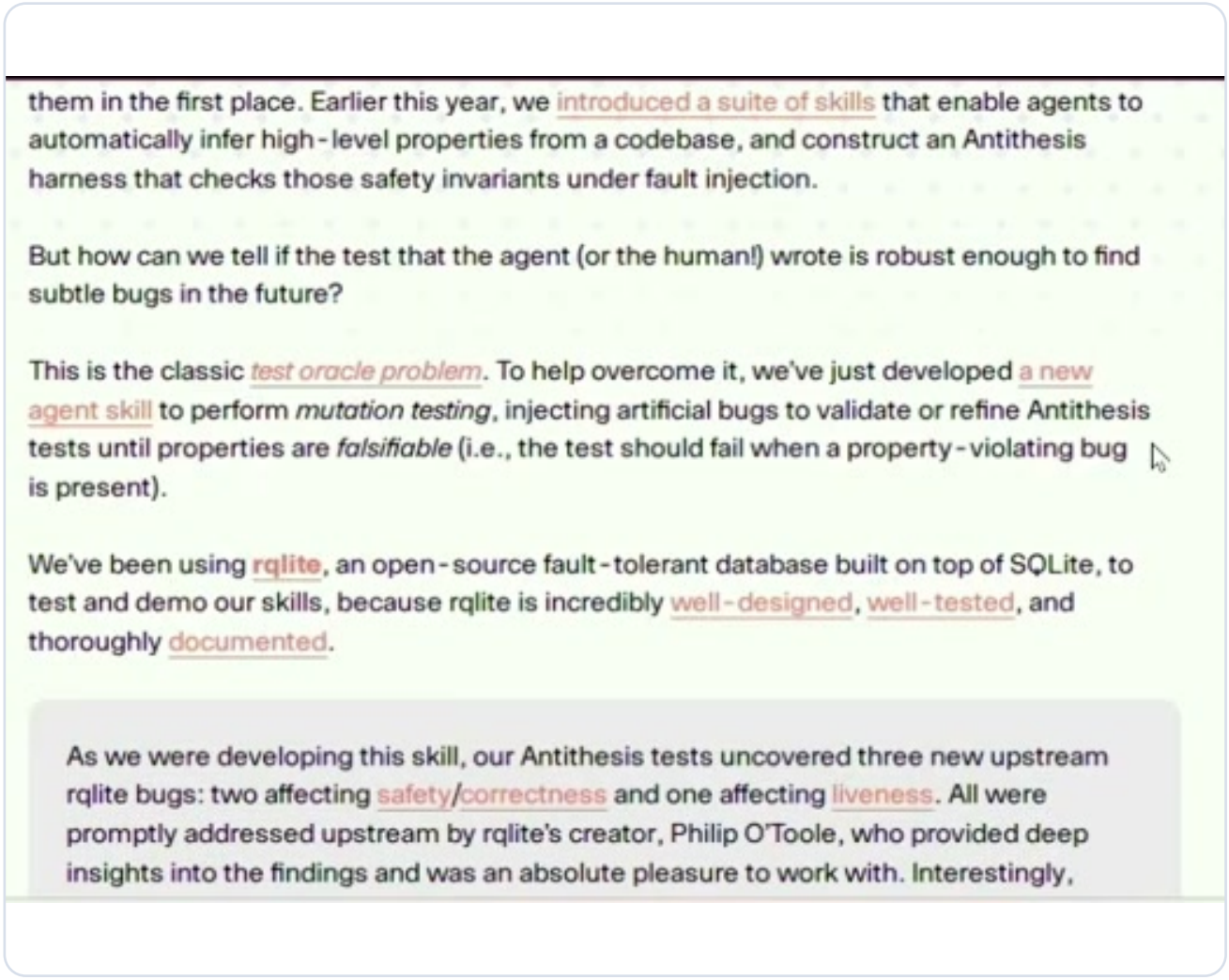
- Mutation testing deliberately injects faults to check whether a property assertion turns red.
- Ordinary operator changes may only cause obvious crashes rather than subtle distributed-system failures.
- The described approach injects race conditions that usually behave correctly but fail in edge cases.
- A property that cannot be falsified may indicate a weak harness or a property needing expert reconsideration.

CORE IDEA

Tests become more credible when realistic injected faults prove they can detect violations.

Source: https://www.youtube.com/watch?v=sTeHIfcv7yE

Mutation campaigns evaluate the tests themselves



them in the first place. Earlier this year, we introduced a suite of skills that enable agents to automatically infer high-level properties from a codebase, and construct an Antithesis harness that checks those safety invariants under fault injection.

But how can we tell if the test that the agent (or the human!) wrote is robust enough to find subtle bugs in the future?

This is the classic *test oracle problem*. To help overcome it, we've just developed a new agent skill to perform *mutation testing*, injecting artificial bugs to validate or refine Antithesis tests until properties are *falsifiable* (i.e., the test should fail when a property-violating bug is present).

We've been using rqlite, an open-source fault-tolerant database built on top of SQLite, to test and demo our skills, because rqlite is incredibly well-designed, well-tested, and thoroughly documented.

As we were developing this skill, our Antithesis tests uncovered three new upstream rqlite bugs: two affecting safety/correctness and one affecting liveness. All were promptly addressed upstream by rqlite's creator, Philip O'Toole, who provided deep insights into the findings and was an absolute pleasure to work with. Interestingly,

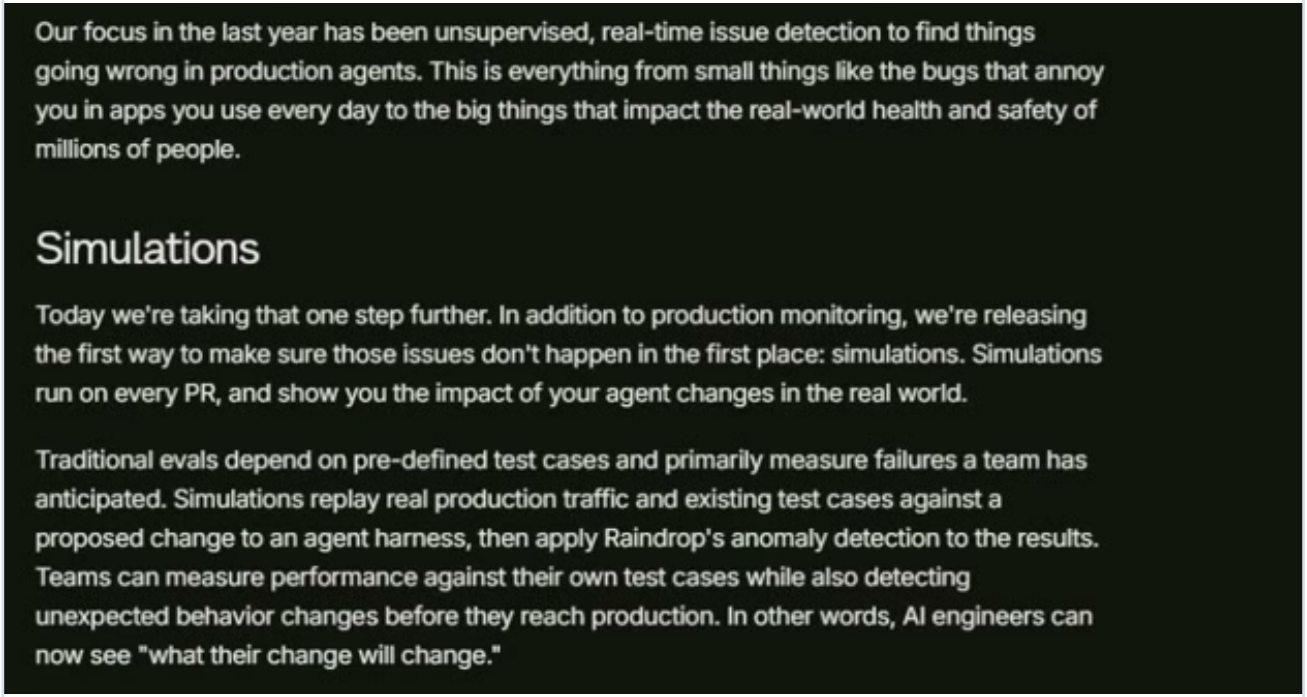
NOTES

- The reported case began with 13 safety properties.
- It ran 19 mutation injections across 46 test runs with about 24 hours of fuzzing.
- Eleven properties were reportedly falsified; two remained unresolved between test setup and property quality.
- The callout reports three newly found rqlite bugs, including safety, correctness, and liveness issues.

CORE IDEA

Planting realistic failures reveals whether the verification system can detect what it claims to protect.

Replay production traffic before an agent change ships



NOTES

- Simulations are described as running on every pull request.
- They replay production traffic and existing tests against a proposed agent-harness change.
- Anomaly detection searches for behavioral differences that predefined evaluations did not anticipate.
- The workflow is described conceptually; the frame does not show a live simulation or measured result.

CORE IDEA

Move production-like behavioral comparison earlier in the delivery cycle.

Autonomous delivery can outpace production visibility



INNOVATION

Enterprise AI Is Moving Fast And Flying Blind

By Bernd Greifeneder, Forbes Councils Member.
for Forbes Technology Council, COUNCIL POST | Membership (fee-based)

Published Sep 10, 2026, 12:00pm EDT, Updated Sep 24, 2026, 10:31am EDT

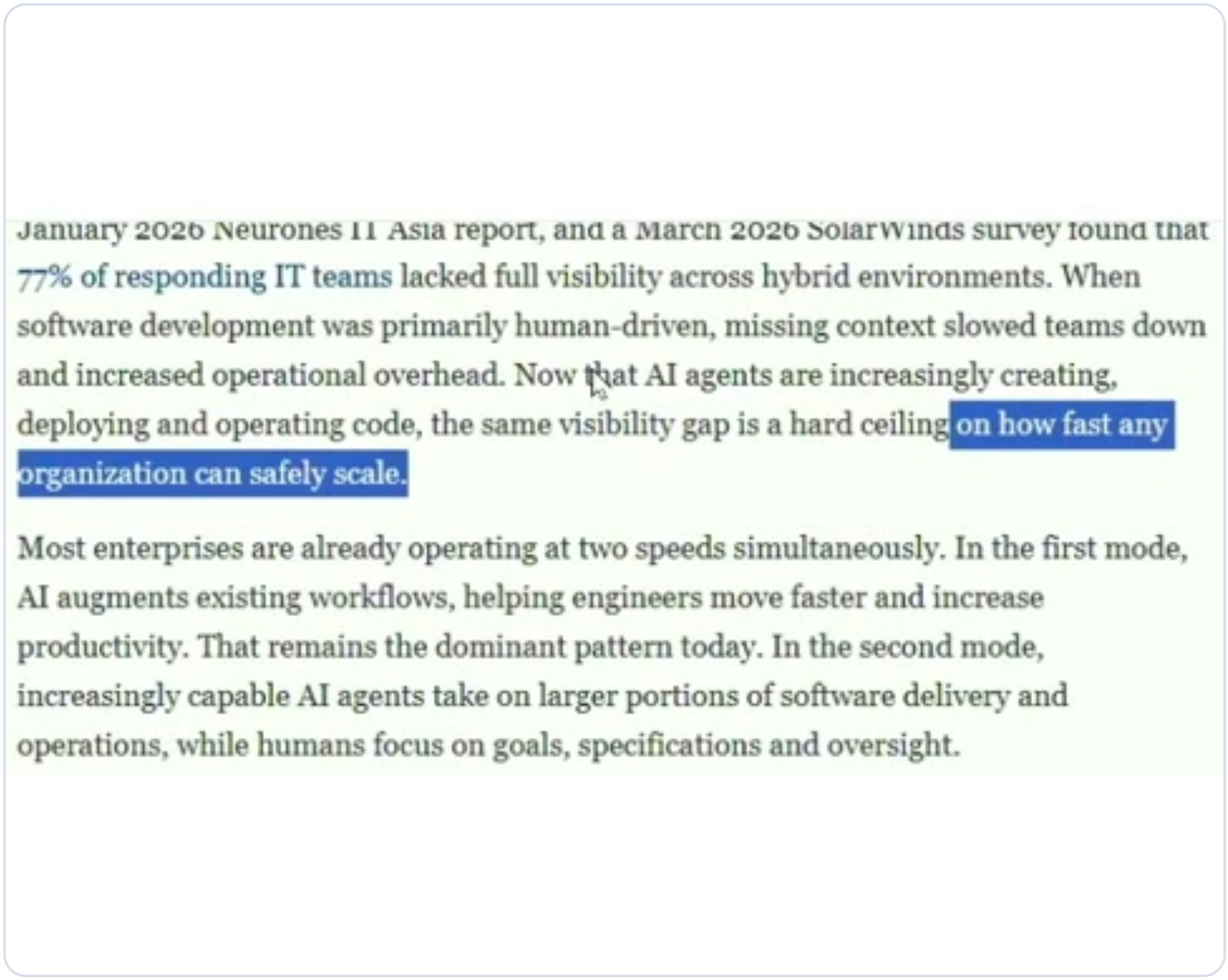
NOTES

- The article argues that agent-driven delivery is advancing faster than production visibility.
- It is a fee-based Forbes Council contribution rather than independent Forbes reporting.
- The author has a commercial observability affiliation through Dynatrace.
- Its conclusions and statistics should therefore remain attributed to the post and cited surveys.

CORE IDEA

Faster autonomous delivery increases risk when teams cannot observe resulting production behavior.

Observability sets a ceiling on safe agent scaling



January 2026 Neurones IT Asia report, and a March 2026 SolarWinds survey found that 77% of responding IT teams lacked full visibility across hybrid environments. When software development was primarily human-driven, missing context slowed teams down and increased operational overhead. Now that AI agents are increasingly creating, deploying and operating code, the same visibility gap is a hard ceiling on how fast any organization can safely scale.

Most enterprises are already operating at two speeds simultaneously. In the first mode, AI augments existing workflows, helping engineers move faster and increase productivity. That remains the dominant pattern today. In the second mode, increasingly capable AI agents take on larger portions of software delivery and operations, while humans focus on goals, specifications and oversight.

NOTES

- The post reports that 77% of surveyed IT teams lacked full visibility across hybrid environments.
- A separate cited report is used for the claim that fewer than one in ten enterprise applications is fully observable.
- The highlighted conclusion is that weak visibility limits safe AI-enabled scaling.
- These statistics come through a commercially affiliated Council Post and should be checked against the original surveys.

CORE IDEA

End-to-end visibility is a prerequisite for governing compounded risk across chained agents.

Source: https://www.youtube.com/watch?v=sTeHIfcv7yE